Database Design Proposal

for the

Complaint Management and Tracking System (CMTS)

NAMAL UNIVERSITY MIANWALI

Department of Computer Science

CSC-271 — Database Systems

Complex Computing Problem — Milestone 1 of Semester Project

Course Name	CSC-271 Database Systems
Instructor	Ms. Asiya Batool
Submission Date	12th March, 2026

Submitted By:

Name	**Registration Number**
Arfa Tayyabah	NUM-BSCS-2024-16
Samra Zamurd	NUM-BSCS-2024-72
Hafsa Gul	NUM-BSCS-2023-24

Contents

List of Tables **ii**

1 Introduction **1**

2 Problem Statement **2**

3 Objectives **3**

4 Scope **4**

4.1 What is Included in Scope 4

4.2 What is NOT Included in Scope 5

5 Data Collection **6**

6 Technology Stack **7**

7 System Functionality **8**

8 User Classes **10**

9 Conclusion **12**

10 Task Division **13**

A AI Prompts **15**

List of Tables

3.1 System Objectives 3

5.1 Data Sources and Collection Methods 6

6.1 Technology Stack with Justification 7

7.1 Key System Functional Requirements 8

1. Introduction

The Complaint Management and Tracking System (CMTS) is a comprehensive platform designed to digitalize the process of submitting, tracking, and resolving complaints within Namal University Mianwali. The system serves as a centralized platform that connects students, faculty, staff, and administrators, enabling efficient communication and streamlined workflow for complaint resolution.

From a database perspective, this system is inherently data-intensive, handling various types of information including user profiles, complaint details, attachments, comments, status updates, feedback, and notifications. The database plays a critical role in ensuring that all this data is stored securely, retrieved efficiently, and managed with integrity. The system must handle thousands of complaints over time, with multiple users accessing and updating data concurrently.

The real-world relevance of this system lies in its ability to replace the current manual, paper-based, or email-based complaint handling process with a structured, data-driven approach. A well-designed database ensures that all complaint-related information is organized, consistent, and easily accessible for both users and administrators. This not only improves the efficiency of complaint resolution but also enables data analysis for performance tracking and continuous improvement.

2. Problem Statement

In the current manual complaint handling system within the university, complaints are submitted via emails, phone calls, or verbal communication. There is no centralized system to record complaints and track their progress. This unstructured approach leads to several critical data management problems.

Complaint data is scattered across multiple sources including emails, paper forms, and informal conversations. There is no single repository where all complaints are stored and managed.The same information is often recorded multiple times in different places, leading to duplication. Without a centralized system, there is no single source of truth, resulting in inconsistent data across different records. Relationships between different pieces of information such as which user submitted which complaint, which attachments belong to which complaint, or which feedback corresponds to which resolution - are not properly maintained. This leads to incomplete or incorrect data.

Generating reports on complaint resolution times, departmentwise performance, or user satisfaction is extremely difficult and time-consuming with manual records. Administrators cannot easily analyze trends or identify areas for improvement.As the university grows, the volume of complaints increases. The current manual system cannot scale to handle hundreds or thousands of complaints efficiently without compromising data quality and accessibility.These data management problems directly impact the efficiency of complaint resolution, user satisfaction, and the ability of administrators to make data-driven decisions. A well-designed database system is essential to address these challenges.

3. Objectives

The main objectives of the proposed database system for the Complaint Management and Tracking System (CMTS) are:

Table 3.1: System Objectives

#	**Objective**
O1	Design a 3NF relational database with well-defined tables for users, complaints, departments, attachments, comments, notifications, and audit records, providing a single centralized platform for all complaint data.
O2	Store the complete complaint lifecycle from initial submission through operator assignment to final resolution or rejection, ensuring end-to-end traceability so that no complaint is ever lost or left unaccounted for.
O3	Enforce data integrity through primary keys, foreign keys, and field-level constraints, and maintain structured audit logs that record every status change with the responsible handler and timestamp, ensuring clear accountability at every stage of the complaint process.
O4	Store comment per complaint with a full activity history, and record notification events with per-user read/unread state, enabling continuous communication between complainants and operators and keeping all parties informed of status changes automatically.
O5	Store post-resolution feedback with satisfaction ratings on a 1–5 scale, and support efficient filtering and pagination queries by complaint status, priority, department, and date range, enabling administrators to identify high-volume departments and slow response times for continuous service improvement.
O6	Design the schema to handle at least 1,000 user records, 10,000 complaint records, and 50,000 file attachments within defined storage constraints while maintaining responsive query performance as the system scales.

4. Scope

The scope of this database project defines the data that will be stored, managed, and processed within the Complaint Management and Tracking System at Namal University Mianwali.

4.1 What is Included in Scope

- **User Data:** Information for all user types including students, faculty, staff, and administrators will be stored. This includes name, email, department, role, registration number, account creation date, and account status.

- **Complaint Data:** Complete complaint records will be stored including subject, description, current status, priority level, room number, and submission timestamp. The history of every status change will also be retained so that the full progression of each complaint is preserved.

- **Department and Tracker Data:** Department names, descriptions, and active status will be stored. Each department will have associated trackers, which are the complaint categories a user selects when submitting a complaint.

- **Attachment Data:** File attachment metadata will be stored for each complaint, including file name, file type, file size, and storage path. The database will not store the actual files, only the information needed to reference them.

- **Comment Data:** Comments made by users and administrators on complaints will be stored along with the author information and the timestamp of each comment.

- **Feedback Data:** After a complaint is resolved, users can submit feedback. The database will store ratings for overall satisfaction, response time, communication quality, and resolution quality, each on a scale of 1 to 5, along with an optional text comment.

- **Notification Data:** Notifications sent to users will be stored along with the notification type, message content, and whether the user has read it or not.

- **Audit Log Data:** A log of all critical actions will be maintained including login

attempts, complaint submissions, and status changes. Each entry will record the user responsible, the action taken, and the time it occurred.

4.2 What is NOT Included in Scope

- The design or implementation of the mobile application interface is not part of this database project.
- Application logic such as complaint routing, report generation, and notification delivery is handled outside the database and is not in scope.
- The internal workings of external systems such as the university authentication service and push notification service are not covered. Only the data exchanged with these systems is stored.
- Anonymous complaint submission is not supported, so no provision for storing complaints without a linked user account is included.

5. Data Collection

Data collection for Complaint Management and Tracking System involves gathering all the information required to efficiently perform the complaint submission and resolution processes. The primary data to be collected is complainant information which will be extracted from University Active Directory. Complaint data will be collected through complaint submission forms. Additional data like complaint handler information, departments and trackers information will be collected from System Administrator.

Table 5.1: Data Sources and Collection Methods

Source	**Description**
Complaint submission forms	When a student, faculty, or staff member fills out form and submits a complaint through the app.
User registration / login	When university members create accounts or log in, their profile data enters the system.
File uploads	When a user attaches a photo as evidence during complaint submission
Feedback forms	When a user rates their experience after their complaint is resolved
Administrator actions	When an administrator assigns, updates, or resolves a complaint, that activity generates status and audit records

6. Technology Stack

The following table outlines the technologies selected for the CMTS database system:

Table 6.1: Technology Stack with Justification

Layer	**Technology**	**Justification**
Database Management System	PostgreSQL 15+	Open-source relational DBMS with support for ENUM types, JSON fields, GIN indexes for full-text search, and concurrent access. Suitable for large complaint datasets.
ORM / Data Access	Prisma (Node.js) or Sequelize	Generates schema migrations automatically, reduces SQL errors, and keeps application models in sync with the database.
Backend Server	Node.js + Express.js	Lightweight, event-driven server that supports REST API communication with the JavaScript-based mobile application.
Notifications	Firebase Cloud Messaging (FCM)	Delivers push notifications for complaint updates and assignments on both Android and iOS. Device tokens stored in the user table.
Dev Tools	pgAdmin 4 / DBeaver	GUI tools for schema visualization, query testing, and data inspection during development.
Version Control	GitHub	Source code versioning and team collaboration with commit history for academic evaluation.
API Testing	Postman	Validates API endpoints and request/response structure before mobile integration.

7. System Functionality

The following functional requirements are directly supported by the database schema and are drawn from Chapter 3 of the Software Requirements Specification (SRS). Each requirement maps to one or more database tables that persist, retrieve, or manage the relevant data, ensuring that all system operations are backed by a structured and consistent data layer.

Table 7.1: Key System Functional Requirements

#	**Functionality**	**SRS Ref**
1	The database shall store user account records for all roles (Student, Faculty, Staff, Operator, Administrator), including university credentials, role, department and account status.	[SRS §3.1.1.1]
2	The database shall enforce an account lockout by recording failed login attempts per user and setting account status to Locked after five consecutive failures.	[SRS §3.4.1.1]
3	The database shall store all active university departments and their respective complaint trackers (categories), including name, description, and active status, used for assigning complaints to the correct handling team.	[SRS §3.1.2.1, 3.1.2.2]
4	The database shall store each complaint record with a system-generated unique ID, subject (max 200 characters), description (max 2,000 characters), priority, status, room number, submission timestamp, and complainant identity.	[SRS §3.1.2.3]
5	The database shall constrain complaint status to a defined enumeration (New, Under Review, In Progress, Resolved, Rejected, Closed), record every status transition with a timestamp and responsible operator, and store the assignment of each complaint to a specific handler.	[SRS §3.1.3.1]

Continued on next page...

#	**Functionality**	**SRS Ref**
6	The database shall store attachment metadata for uploaded images, enforcing a maximum of five files per complaint and a maximum file size of 2 MB each, with only JPG and PNG types permitted.	[SRS §3.1.2.3]
7	The database shall persist all complaint comments with the comment body (max 1,000 characters), author identity, and timestamp.	[SRS §3.1.4.1]
8	The database shall store exactly one feedback record per resolved complaint, comprising a mandatory overall rating (1–5 stars) and optional ratings for response time, communication quality, and resolution quality.	[SRS §3.1.5.2]
9	The database shall support filtering complaints by status, priority, and date range, and keyword search across complaint subject and description fields, requiring appropriate indexing on those columns.	[SRS §3.1.3.3–3.1.3.7]
10	The database shall support aggregated queries for reporting, including complaint counts by status, priority, department, and time period, as well as average resolution times and satisfaction rating averages.	[SRS §2.2.4]

8. User Classes

The system implements role-based access control (RBAC) with two distinct user classes, each assigned a corresponding database role that scopes their permitted operations. This design ensures that users can only read or write data relevant to their function, in accordance with the access control requirements defined in SRS §2.3 and §3.4.1.3.

End Users (Students, Faculty, and Staff)

End users represent the primary complainants of the system. This class includes undergraduate students, teaching faculty, and administrative and technical staff, giving a combined expected volume of 450–620 users (SRS §2.3.1–2.3.3). All three sub-groups share the same database role `app_user` and are granted identical access permissions. Their permitted operations are:

- Submit new complaint records and read only their own submissions.
- Add comments to their own complaints and upload image attachments as evidence.
- Submit feedback on their own resolved complaints.
- Read their own notification records and mark them as read.

End users may not read, modify, or delete complaint records belonging to other users. All access is authenticated via university credentials before any database operation is permitted (SRS §3.1.1.1).

System Administrator

System administrators are the complaint handlers and platform managers responsible for the day-to-day operation of the CMTS. Administrators are assigned the database role `app_admin` and are granted following permissions:

- Read all complaint records across all departments and all users.
- Update complaint status, assign complaints to operators, and add comments on any complaint.
- Configure departments, trackers, and complaint categories.
- Manage user accounts, including unlocking locked accounts.

- Generate and export performance reports and analytics.
- All of the above operations are recorded in the audit log without exception (SRS §3.4.1.2).

9. Conclusion

This proposal has outlined the major system requirements for the database layer of the Complaint Management and Tracking System (CMTS). The requirements have been derived directly from the Software Requirements Specification (SRS) dated 28 December 2025 and cover the full scope of data the system must store, the constraints it must enforce, and the operational requirements it must meet.

The proposed database system is designed to support a structured and transparent complaint lifecycle, from initial submission by students, faculty, and staff through operator-managed resolution, feedback collection, and performance reporting. This proposal serves as the baseline and as a reference point to validate that the database design remains aligned with the goals of the CMTS project.

10. Task Division

At this stage of the project, a formal division of tasks among team members has not yet been decided. Since the project is still in its early phases, the actual database schema has not been finalized, and the full scope of implementation work remains undefined. Once the schema is finalized and the implementation requirements become clearer, tasks will be divided among team members according to each member's strengths and the complexity of each component.

References

[1] Arfa Tayyabah, Samra Zamurd, and Hafsa Gul, "Software Requirements Specification: Complaint Management and Tracking System," Department of Computer Science, Namal University Mianwali, CSC-225 Software Engineering, Dec. 2025.

[2] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," *postgresql.org*, 2023. [Online]. Available: `https://www.postgresql.org/docs/15/`

[3] Prisma, "Prisma ORM Documentation," *prisma.io*. [Online]. Available: `https://www.prisma.io/docs`

[4] OpenJS Foundation, "Express.js Documentation," *expressjs.com*. [Online]. Available: `https://expressjs.com`

[5] Google LLC, "Firebase Cloud Messaging Documentation," *firebase.google.com*. [Online]. Available: `https://firebase.google.com/docs/cloud-messaging`

[6] Arfa Tayyabah, Samra Zamurd, and Hafsa Gul, "CMTS Database Systems Project Repository," *github.com*, accessed Mar. 2026. [Online]. Available: `https://github.com/arfa-tayyabah/Complaint-Management-and-Tracking-System`

A. AI Prompts

The following table documents all AI tools used during the preparation of this proposal, including the purpose, prompt used, and date of access.

[1] Claude (Anthropic)," Below are the objectives of database design proposal for complaint management and tracking system project. Help me to make them measurable and in clear terms and keep the tone simple," *claude.ai*, accessed Mar. 9, 2026.

[2] Claude (Anthropic), "I am attaching my SRS document for complaint management and tracking system. Based on the system requirements, what technologies would you recommend for the database, backend, mobile app and other necessary components," *claude.ai*, accessed Mar. 10, 2026.

[3] Deepseek, "I have attached my software engineering project's SRS document, along with that, I also have the PDF for my next milestone, in which we have to work on the same previous project, but there will be some changes for Database Systems. From these attached documents, tell me what will be included for database systems and what changes need to be made, etc." *claude.ai*, accessed Mar. 9, 2026.

[4] Claude (Anthropic), "Evaluate proposal document against given milestone instructions and evaluation rubric," *claude.ai*, accessed Mar. 11, 2026.

[5] Claude (Anthropic) "Act as senior database manager with years of experience in industry, analyze the given CCP document. Write foundational database design report template in Latex. The latex template should include proper formatting like titles, sections, title page etc." *claude.ai*, accessed Mar. 8, 2026.